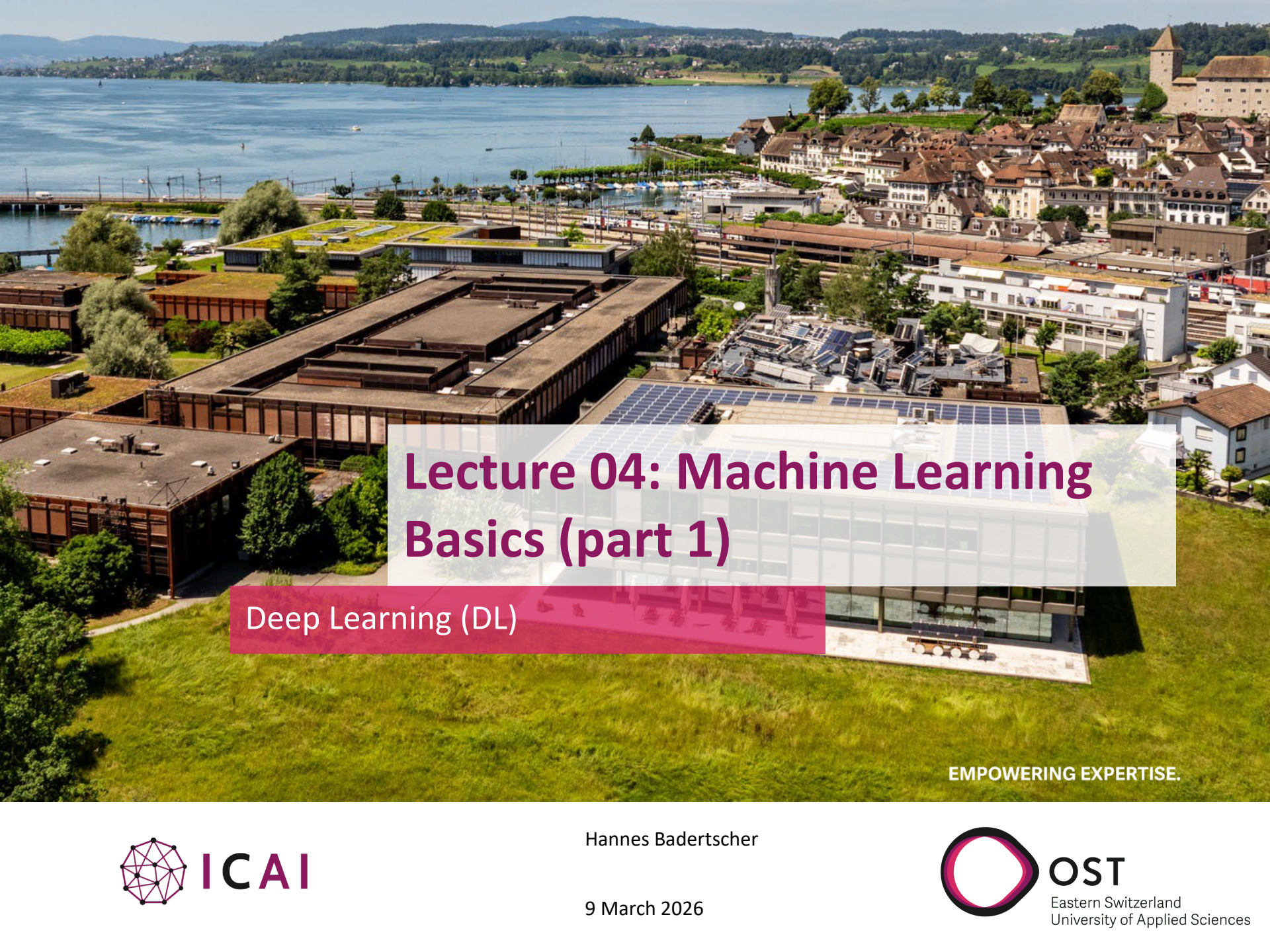




Lecture 04: Machine Learning Basics (part 1)

Deep Learning (DL)

EMPOWERING EXPERTISE.

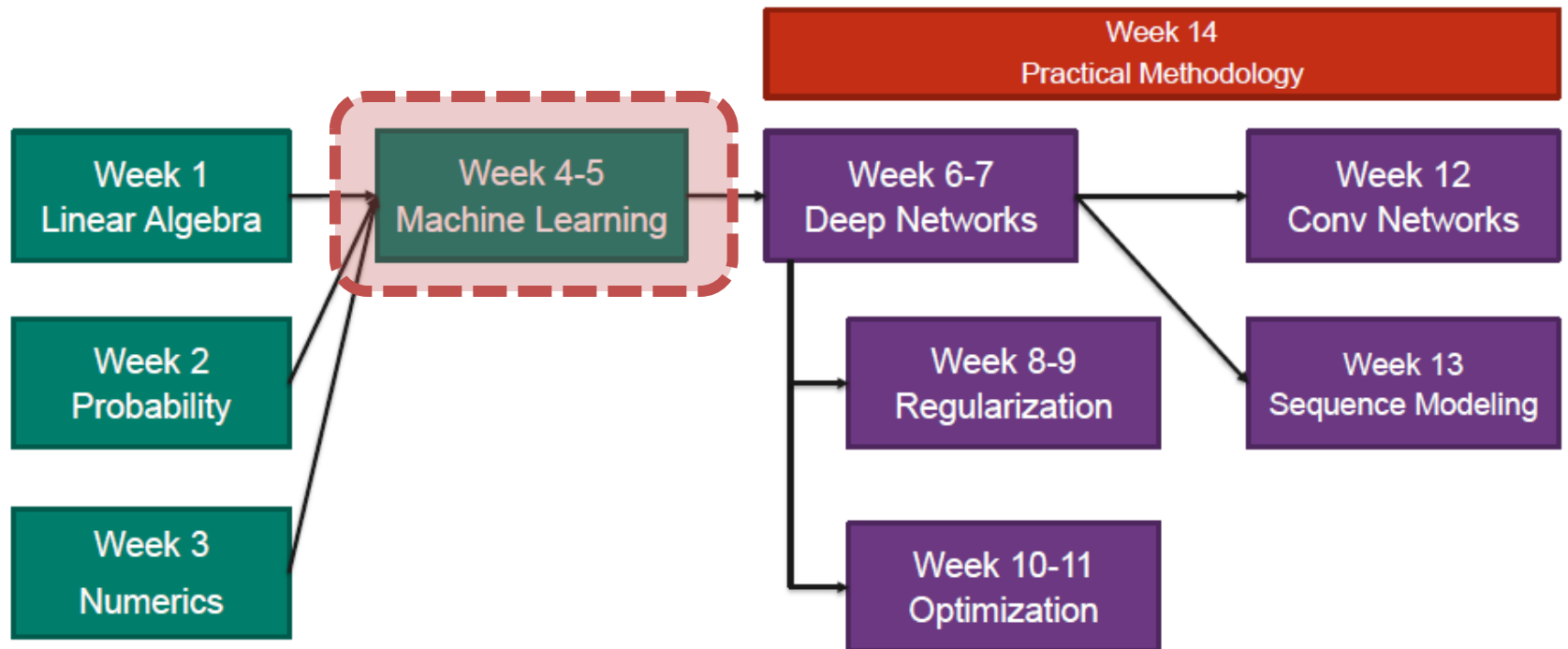


Hannes Badertscher

9 March 2026



Overview of DL-Lecture



Recap of Week 1 - 3

- Given a set of m **training data** points
- The **empirical data distribution** is then:
- The **neural network predicts** a distribution
 - Where θ are the learnable parameters
- **Training process:** make $p_{\text{model}}(\mathbf{x})$ as similar as possible to $\hat{p}(\mathbf{x})$ by **minimizing the KL-divergence**
- The KL-divergence can be minimized numerically with **Gradient Descent**
- ➔ **How to do Machine Learning with this?**

$$\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$$

$$\hat{p}(\mathbf{x}) = \frac{1}{m} \sum_{i=1}^m \delta(\mathbf{x} - \mathbf{x}^{(i)})$$

$$p_{\text{model}}(\mathbf{x}; \boldsymbol{\theta})$$

$$D_{\text{KL}}(P \| Q)$$

$$\mathbf{x}' = \mathbf{x} - \epsilon \nabla_{\mathbf{x}} f(\mathbf{x})$$

Overview of this lecture

- Example: Linear Regression

- Capacity, Overfitting and Underfitting

- Estimators, Bias and Variance

Week 4

- Maximum Likelihood Estimation

Week 5

- Stochastic Gradient Descent

- Building a Machine Learning Algorithm

- Challenges Motivating Deep Learning

Example: Linear Regression

Definition:

A machine learning algorithm improves a computers performance P at some task T via experience E .

- Concrete example: **Linear Regression**

- Input vector: $\mathbf{x}$
- Output: y
- **Goal:** predict y from $\mathbf{x}$
- **Assumption:** linear relationship between $\mathbf{x}$ and y
- Parameters $\mathbf{w}$: degrees of freedom
- ➔ Learning: find these parameters

$$y \in \mathbb{R} \quad \mathbf{x} \in \mathbb{R}^n$$

$$\hat{y} = \mathbf{w}^\top \mathbf{x} \quad !$$

- The **task T**: predict y from $\mathbf{x}$

Example: Linear Regression

A machine learning algorithm improves a computers performance P at some task T via experience E .

- **Performance measure P**
 - **Overall goal: predict y** as well as possible when applying linear regression to **new data**
 - ➔ Measurement using a **separate test data** set
 - Possible performance measure: **mean squared error (MSE)**

$$\mathbf{X}^{(\text{test})} \quad \mathbf{y}^{(\text{test})}$$

$$\begin{aligned} \text{MSE}_{\text{test}} &= \frac{1}{m} \sum_i (\hat{\mathbf{y}}^{(\text{test})} - \mathbf{y}^{(\text{test})})_i^2 \\ &= \frac{1}{m} \|\hat{\mathbf{y}}^{(\text{test})} - \mathbf{y}^{(\text{test})}\|_2^2 \end{aligned}$$

!

Example: Linear Regression

A machine learning algorithm improves a computers performance P at some task T via experience E .

- **Experience E**
 - “experience” means data → learn from data
 - Data for learning → **training data set**

$$(\mathbf{X}^{(\text{train})}, \mathbf{y}^{(\text{train})})$$

- → **Summary of linear regression**
 - **Task T** : predicting y based on $\mathbf{x}$
 - **Experience E** : training data
 - **Performance P** : test MSE

$$\hat{y} = \mathbf{w}^\top \mathbf{x}$$

$$(\mathbf{X}^{(\text{train})}, \mathbf{y}^{(\text{train})})$$

$$\text{MSE}_{\text{test}} = \frac{1}{m} \|\hat{\mathbf{y}}^{(\text{test})} - \mathbf{y}^{(\text{test})}\|_2^2$$

Example: Linear Regression

Goal: minimize MSE_{test}

- **Problem:** for learning, we only have

$$(\mathbf{X}^{(\text{train})}, \mathbf{y}^{(\text{train})})$$

- **Solution:** minimize MSE_{train} instead

- Note: this is not the same thing!
- But: intuitively makes sense

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} MSE_{train}$$

- For linear regression, this can be solved analytically:

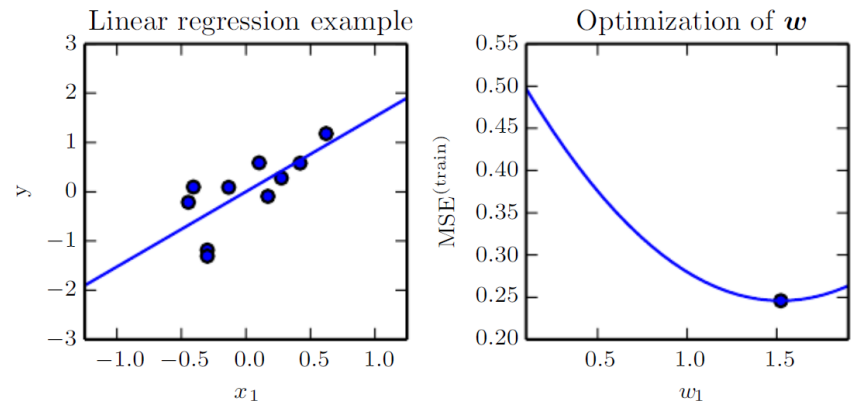
- Calculate gradient, set to 0, solve for $\mathbf{w}$
- Note: in DL, this is not possible!

$$\nabla_{\mathbf{w}} MSE_{train} = 0$$

$$\Rightarrow \mathbf{w} = \left(\mathbf{X}^{(\text{train})\top} \mathbf{X}^{(\text{train})} \right)^{-1} \mathbf{X}^{(\text{train})\top} \mathbf{y}^{(\text{train})}$$

Example: Linear Regression

- Example with 1-d data
 - x_1 : measured input data
 - y : measured corresponding output
 - Linear equation: $\hat{y} = w_1 x_1$
 - Right: training MSE vs different values for w_1



DL, Fig. 2.3

- Note:
 - The analytical solution directly allows to calculate the solution $w_1 = 1.5$
 - Gradient descent on MSE_{train} would lead to the same result

Example: Linear Regression

- Previous examples used no intercept (bias term) for simplicity
 - $\rightarrow \hat{y} = 0$ for $x = 0$
- **Solution:** add an **intercept / bias term** b to the linear model
- **Typical implementation:** add a column of 1's to $\mathbf{X}$ instead
 - $\rightarrow$ No need to complicate the equation and treat $\mathbf{w}$ and $\mathbf{b}$ separately
- **Note:** the bias term is not the same as the concept of statistical bias (that is discussed later today)

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

$$\mathbf{X} = \begin{bmatrix} x_{11} & 1 \\ x_{12} & 1 \\ x_{13} & 1 \end{bmatrix}$$

Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Week 4

Week 5

Capacity, Overfitting and Underfitting

Goal of Machine Learning
Perform well on new, unseen data (test data)

- **Goal:** achieve a low MSE_{test}
- **Problem:** we train on MSE_{train} and not on MSE_{test}
- ➔ train on MSE_{train} and “*hope for the best*”?
- ➔ is this reasonable?

$$MSE_{test} = \frac{1}{m^{(test)}} \| \mathbf{X}^{(test)} \mathbf{w} - \mathbf{y}^{(test)} \|_2^2$$

$$MSE_{train} = \frac{1}{m^{(train)}} \| \mathbf{X}^{(train)} \mathbf{w} - \mathbf{y}^{(train)} \|_2^2$$

Capacity, Overfitting and Underfitting

Why is it ok to **train on the $\text{MSE}_{\text{train}}$** when we actually want a **small MSE_{test}** ?

- The **i.i.d. assumptions**
 - Examples (samples, data points) are **independent** of each other
 - Training set and test set are **identically distributed**
- ➔ both training and test data is generated by the same **data generation process** given as
- In that case:

- Independent
- Identically distributed

!

p_{data}

$$E[\text{MSE}_{\text{train}}] = E[\text{MSE}_{\text{test}}] \quad !$$

Capacity, Overfitting and Underfitting

- Under the i.i.d. assumptions,
- → To make $E[\text{MSE}_{\text{test}}]$ small, we can do two things:

$$E[\text{MSE}_{\text{train}}] = E[\text{MSE}_{\text{test}}]$$

1. Make $\text{MSE}_{\text{train}}$ small.
2. Keep gap between $\text{MSE}_{\text{train}}$ and MSE_{test} small.

- **Capacity** of a model: “power” or **flexibility** to fit the training data
- Too **small capacity** → fail to make $\text{MSE}_{\text{train}}$ small → **underfitting**
- Too **large capacity** → gap between $\text{MSE}_{\text{train}}$ and MSE_{test} increases → **overfitting**

Capacity, Overfitting and Underfitting

- **Example: Linear regression**

- Standard linear model:

$$\hat{y} = b + wx$$

- Extension to **increase capacity**: add **quadratic** term

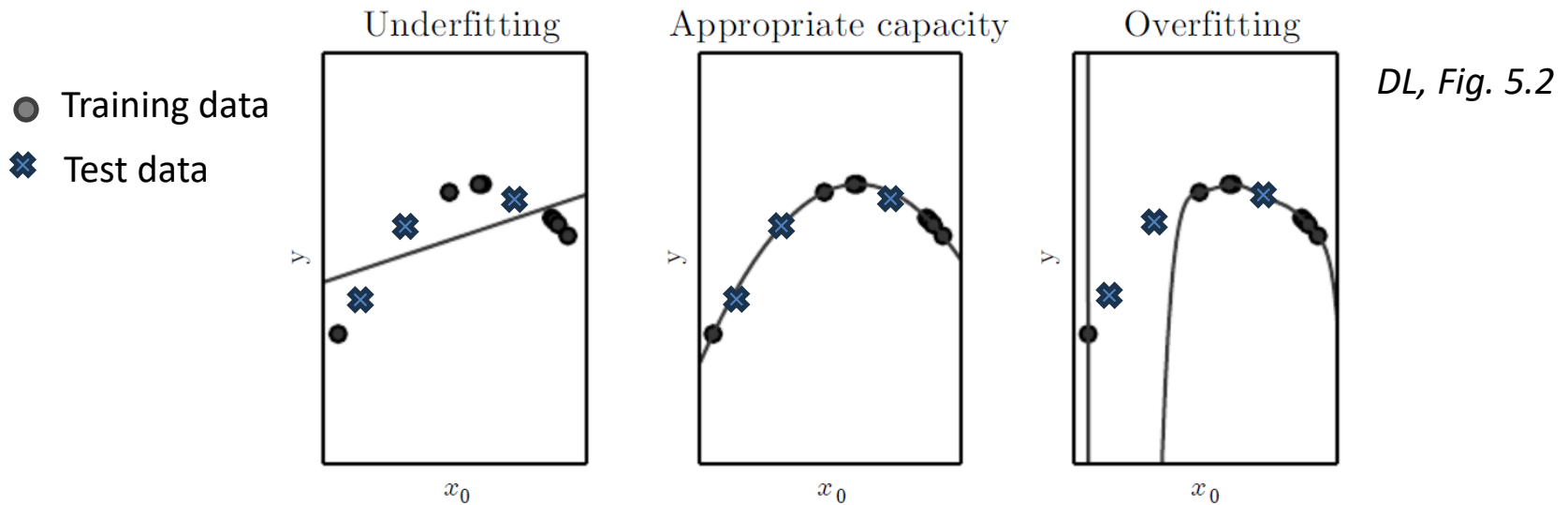
$$\hat{y} = b + w_1x + w_2x^2$$

- Further **increase in capacity**: add **higher-order polynomial** terms

$$\hat{y} = b + \sum_{i=1}^9 w_i x^i$$

- The number of learnable parameters increases from 2 (linear) to 3 (quadratic) to 10 (polynomial of 9. order)

Capacity, Overfitting and Underfitting



Insufficient capacity

- Training and test error (too) large
- Not complex enough to capture underlying p_{data}

Appropriate capacity

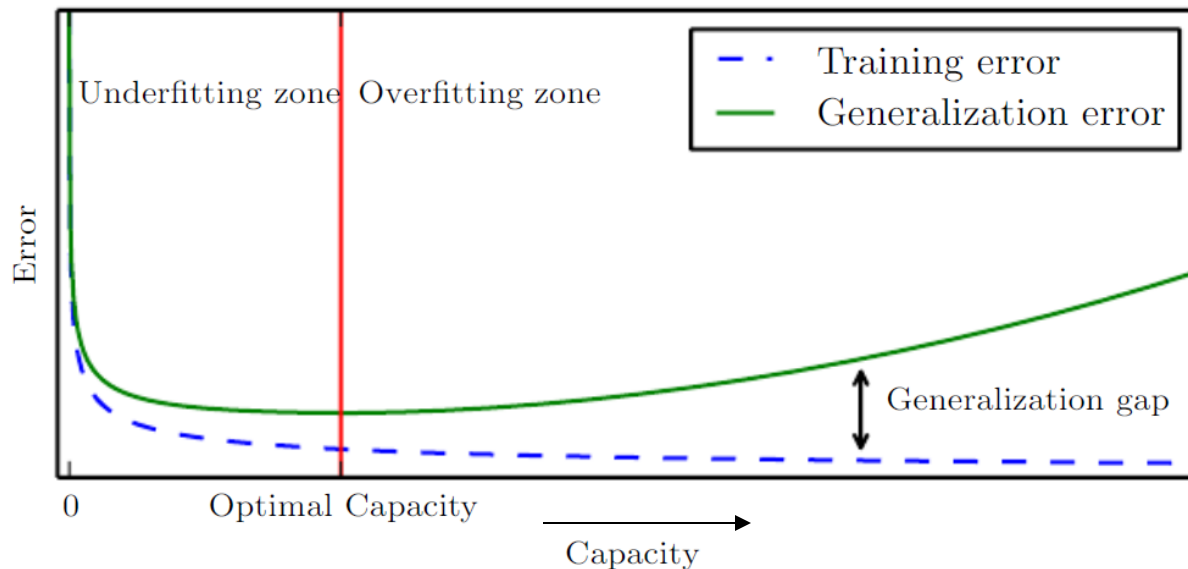
- Small training and test error
- Model capacity matches complexity of data

Excessive capacity

- Training error $\rightarrow 0$
- Large gap between training and test error
- Bad generalization

Capacity, Overfitting and Underfitting

- Capacity v.s. error rate
 - **Training error always decreases** with capacity
 - **Test error follows a U-shape**



DL, Fig. 5.3

Capacity, Overfitting and Underfitting

- If the true data generating distribution p_{data} is known, a **perfect classifier** can be built

p_{data}

- Reality: only $\hat{p}_{\text{data}}$ known

- But: resulting **test error rate** will (typically) **not be 0!**

$$\text{MSE}_{\text{test}} > 0$$

- Sources of error: e.g. measurement errors, missing relation between x and y , ...

- Resulting (test) error of a perfect classifier = “**Bayes error rate**” or “**irreducible error**”

The No Free Lunch Theorem

Definition
(easy):

No machine learning algorithm is universally any better than any other.

Definition:
(complicated)

Averaged over all possible data generating distributions, **every classification algorithm has the same error rate** when classifying previously unobserved points.

- **Bad news? No!**
 - “when averaged **over all possible data generating distributions**”
 - Typically, we tailor a machine learning problem towards a certain kind of data generating distribution
 - The goal is *not* to find a universal learning algorithm.
 - ➔ we *can* find the **optimal algorithm for our problem**

Regularization

- **No free lunch theorem** → ML algorithm must be tailored to our problem!

→ Find optimal capacity!

- Two options:
 - 1) Find **optimal capacity** by **extending** algorithm
 - 2) Build an algorithm with a **too large capacity**, then guide our algorithm to solutions we prefer
- In **DL** → **option 2 = regularization** has proven more successful

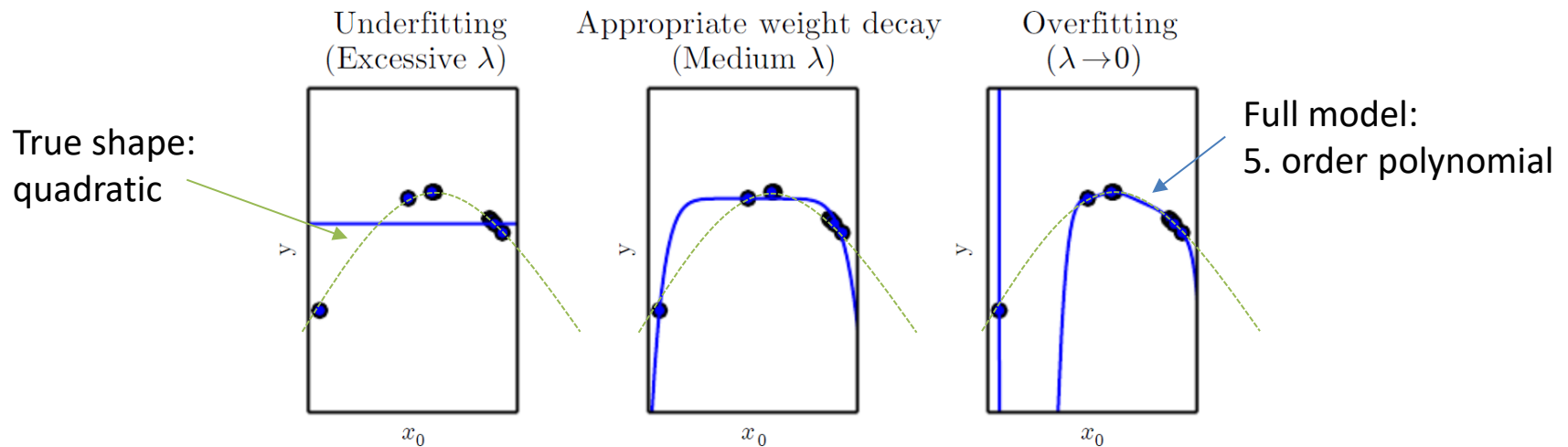
$$\hat{y} = b + \sum_{i=1}^9 w_i x^i$$

Regularization

- **Example:** Polynomial regression with weight decay
 - Approach: select a (constant) very **high model order**
 - Control capacity by enforcing **weights with small Euclidean norm**
 - λ directly controls the **flexibility**

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^\top \mathbf{w}$$

λ	Capacity	Reason
0	High	No regularization
inbetween		
∞	0	$\mathbf{w}^\top \mathbf{w}$ must be 0



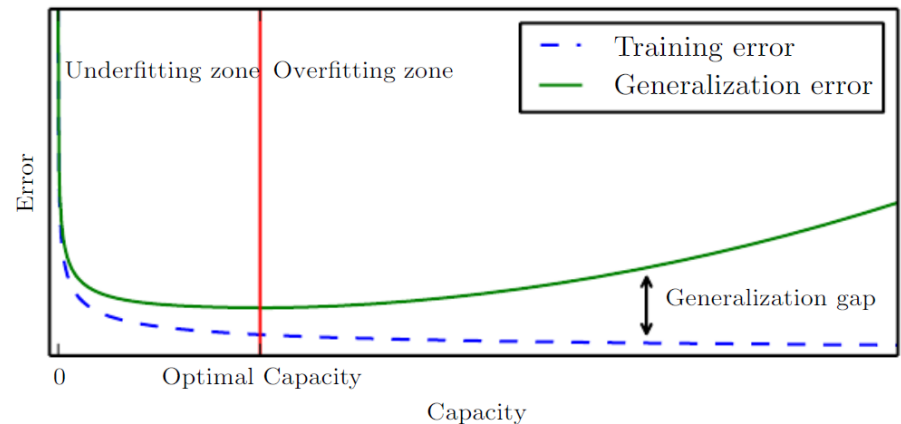
DL, Fig. 5.5

Regularization

Definition:

Regularization is **any modification** we make to a learning algorithm that is intended to reduce **its generalization error but not its training error**

- Regularization always **reduces the capacity** of a model
- Goal: **find optimal capacity** where generalization error is minimal
- Philosophy of deep learning:
 - 1) Build a very large and flexible model
 - 2) Regularize it



DL, Fig. 5.3



Hyperparameters and Validation Sets

- Definition:

Hyperparameters are **parameters** of a learning algorithm that are **not learned** by the algorithm.

- → Hyperparameters must be selected by the developer

- How to select hyperparameters?

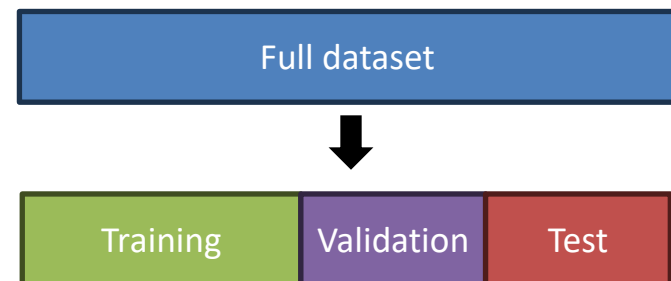
- Using the **training set** → **No**, will choose the highest capacity
- Using the **test set** → **No**, not allowed

- A third dataset is required, the so-called **validation set**

- Typically, a dataset is **split randomly** into e.g. 60% training, 20% validation, 20% testing

Examples:

- Polynomial order
- λ of weight decay



Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Week 4

Week 5

Estimators, Bias and Variance

Goal:

Formalize the ideas of **generalization**, **underfitting** and **overfitting** using statistics

- General statistics: **estimators** which **estimate a value based on data**
 - **Example:** the **sample mean** is an estimator for the true **mean**
 - Estimators can be formally characterized with their **bias** and **variance**.
- **Machine learning** is also an **estimation** problem
 - Instead of estimating a sample mean, we **estimate the parameters of a model**
 - ➔ same mathematical tools can be used

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

Estimators

- “**Point estimate**” → calculates the **best estimate** for a parameter
 - Works for either 1 parameter or a vector of parameters

- **Notation:**

True value of parameter	θ
Estimate	$\hat{\theta}$

- **Estimator:** any function $g(\dots)$ that estimates θ based on measured data
 - Assumption: dataset of m **i.i.d. data samples**
 - $\hat{\theta}_m$: estimate of θ based on m samples

$$\hat{\theta}_m = g(x^{(1)}, \dots, x^{(m)})$$

$$\{x^{(1)}, \dots, x^{(m)}\}$$

Bias

- **Definition of Bias**

- Note: **Expectation** → if we **repeatedly calculate** $\hat{\theta}_m$ many times
- This would require collecting **many datasets** and running the estimator on each → theoretical tool

$$\text{bias}(\hat{\theta}_m) = \mathbb{E}(\hat{\theta}_m) - \theta \quad !$$

- **Definition:** an estimator is **unbiased** if:

- → On average, the estimator finds the correct value

$$\mathbb{E}(\hat{\theta}_m) = \theta \quad !$$

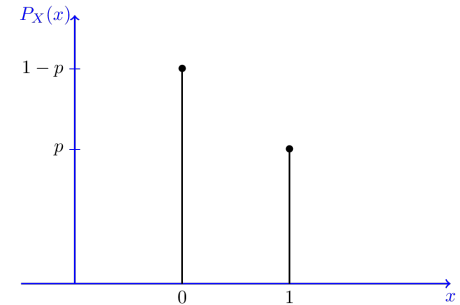
- **Definition:** an estimator is **asymptotically unbiased** if:

- → On average, the estimator finds the correct value – if we use an **infinite amount of training data**

$$\lim_{m \rightarrow \infty} \mathbb{E}(\hat{\theta}_m) = \theta \quad !$$

Example: Bernoulli Distribution

- **Goal:** estimate parameter θ of Bernoulli distribution
- Definition of Bernoulli distribution



$$P(x = 1) = \theta$$

$$P(x = 0) = 1 - \theta$$

- The **expectation** of a Bernoulli random variable is:

$$E[x] = 0 \cdot P(x = 0) + 1 \cdot P(x = 1) = \theta$$

- ➔ The parameter θ can be estimated with the sample mean
- ➔ Is this an unbiased estimator?

$$\hat{\theta}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

Example: Bernoulli Distribution

- Plug in estimator (sample mean) into equation for bias

$$\text{bias}(\hat{\theta}_m) = \mathbb{E}[\hat{\theta}_m] - \theta$$

$$= \mathbb{E} \left[\frac{1}{m} \sum_{i=1}^m x^{(i)} \right] - \theta$$

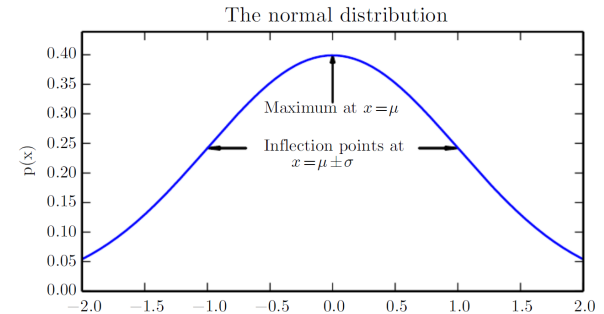
...

$$= \theta - \theta = 0$$

- **→ Yes**, using sample mean for estimating θ in a Bernoulli distribution is **unbiased**

Example: Gaussian Distribution

- **Goal:** estimate parameters μ and σ^2 of Gaussian distribution
- The expectation of the Gaussian is the mean $\mu \rightarrow$ use sample mean to estimate μ



DL, Fig. 3.1

$$E[x] = \mu$$

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

- Again, it can be shown that the **sample mean** is an **unbiased estimator** for estimating the parameter μ of a Gaussian

$$\text{bias}(\hat{\mu}_m) = \mathbb{E}[\hat{\mu}_m] - \mu$$

$$\vdots$$

$$= \mu - \mu = 0$$

- $\rightarrow$ What about the variance σ^2 ?

Example: Gaussian Distribution

- Simple estimator for variance:
replace all **expectations** with **sample means**

$$\text{Var}(f(x)) = \mathbb{E} \left[(f(x) - \mathbb{E}[f(x)])^2 \right]$$

$$\hat{\sigma}_m^2 = \frac{1}{m} \sum_{i=1}^m \left(x^{(i)} - \hat{\mu}_m \right)^2$$

- ➔ Is this an unbiased estimator?
- ➔ No, it is biased = wrong on average
 - But: asymptotically unbiased

$$\text{bias}(\hat{\sigma}_m^2) = \frac{m-1}{m} \sigma^2 - \sigma^2 = -\sigma^2/m$$

- This estimator can **easily be fixed** to be unbiased
- The **unbiased sample variance estimator** is given as

$$\tilde{\sigma}_m^2 = \frac{1}{m-1} \sum_{i=1}^m \left(x^{(i)} - \hat{\mu}_m \right)^2$$

Variance and Standard Error

- **Bias:** how good is your estimate is on average.

- **Variance:** how much does your estimate vary.

$$\text{Var}(\hat{\theta})$$

- **Standard error:** square root of variance

$$\text{SE}(\hat{\theta})$$

- **A good estimator is**
 - **Unbiased**
 - Has a **small variance**

Example: Gaussian Distribution

- **Standard error of sample mean estimator**

- σ = standard deviation of the Gaussian

$$\text{SE}(\hat{\mu}_m) = \sqrt{\text{Var}\left[\frac{1}{m} \sum_{i=1}^m x^{(i)}\right]} = \frac{\sigma}{\sqrt{m}}$$

- **Goal:** small standard error

- **→** is $\sigma/\sqrt{m}$ small or large?

- **Important findings**

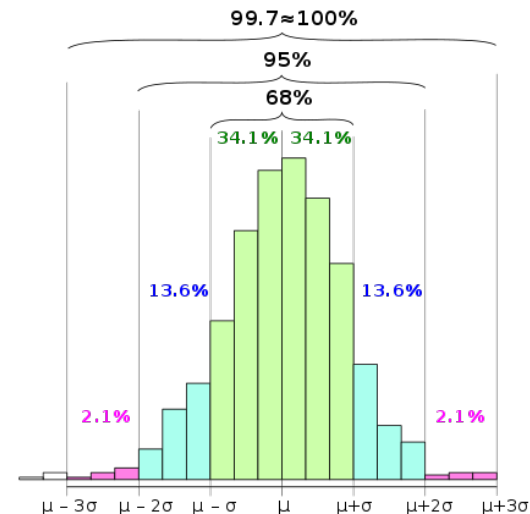
- **→** the **smaller the variance** of a Gaussian, the **smaller the standard error** of the sample mean
- **→** the **more samples** used for estimating the sample mean, the **smaller it's standard error**

Variance and Standard Error

- **Error metrics** are also **estimated** and hence also **have a standard error / variance**
- For example, the **test MSE** is estimated using a **sample mean**
- ➔ the **higher the number of samples** in the test dataset, the **better** the estimate
- ➔ the **uncertainty can be quantified**, e.g. with a **95% confidence interval**

$$CI = [\mu - 2\sigma, \mu + 2\sigma]$$

$$MSE_{\text{test}} = E[\text{squared error}]$$



Consistency

- **Consistency:** combination of Bias and Variance
- Definition of **consistent estimator**:
 - With **infinitely many samples**, the estimator produces **exactly θ**
 - **→** (asymptotically) **unbiased**
 - **→ standard error decreases to 0** with infinitely many samples
- **→ Good estimators** should be **consistent**
- **→ Goal for ML/DL:** find **consistent estimators** for all learnable parameters

$$\lim_{m \rightarrow \infty} \hat{\theta}_m \xrightarrow{p} \theta \quad !$$

Bias-Variance Trade-Off

- How to **compare multiple estimators** (or ML models) with **different biases and variances**?
 - For example:
 - Model 1: low variance, high bias
 - Model 2: low bias, high variance

- → **Mean squared error (MSE)** between **estimated parameter** $\hat{\theta}_m$ and **true** (unknown) θ

$$\text{MSE} = \mathbb{E}[(\hat{\theta}_m - \theta)^2] \quad !$$

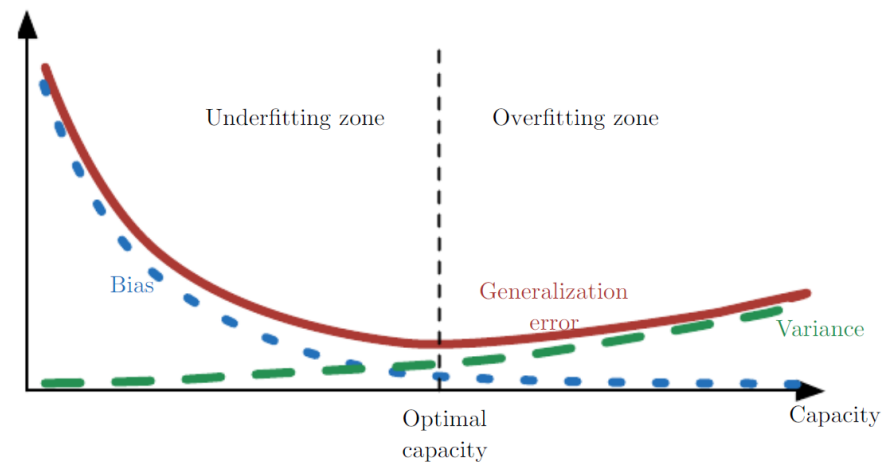
- The MSE can be divided into:

$$= \text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m) \quad !$$

Bias-Variance Trade-Off

- Low capacity → high bias but low variance
- High capacity → low bias but high variance
- → **Medium capacity** with a **good trade-off** between bias and variance
- The **generalization error** can be divided into **bias** and **variance**

$$\text{MSE} = \text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$



DL, Fig. 5.6

Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Week 4

Week 5

Summary:

A machine learning algorithm improves a computers performance P at some task T via experience E .

- **Definition of machine learning algorithm:**

- **i.i.d. assumptions**

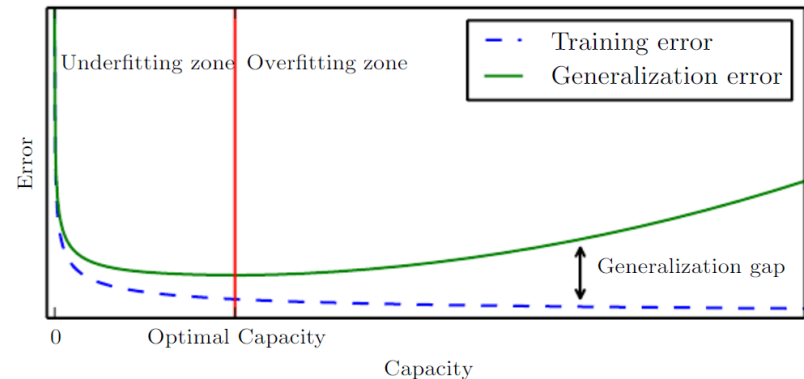
- Independent and identically distributed
- → allows to train on MSE_{train} with the goal of reducing MSE_{test}

$$E[MSE_{\text{train}}] = E[MSE_{\text{test}}]$$

- **Capacity of a model**

- **Regularization**

Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error



- **Bias**

$$\text{bias}(\hat{\theta}_m) = \mathbb{E}(\hat{\theta}_m) - \theta$$

- **Variance**

$$\text{Var}(\hat{\theta})$$

- **Consistency**

$$\lim_{m \rightarrow \infty} \hat{\theta}_m \xrightarrow{p} \theta$$

- **Bias-Variance Trade-Off**

$$MSE = \text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$

Overview of DL-Lecture

